Graphic Era Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

VIRTUAL CITY SIMULATOR

Student/Team Information

Team Name:	**uss.mehul**
Team member 1 (Team Lead) **NAME-:** MEHUL PANGTEY, **UNIVERSITY ROLL NO.-:** 2518462, **STUDENT ID-:** 25101281550, **EMAIL-:** ucss.mehul@gmail.com	
Team member 2 **NAME-:** PARAKRAM BHATT, **UNIVERSITY ROLL NO.-:** 2518503, **STUDENT ID-:** 25101280890, **EMAIL-:** parakrambhatt30@gmail.com	

Team member 3 **NAME-:** ABHINAV CHAUHAN, **UNIVERSITY ROLL NO.-:** 2518120, **STUDENT ID-:** 25101280513, **EMAIL-:** kumarabhinav4545@gmail.com	
Team member 4 **NAME-:** ANJALI NAWANI, **UNIVERSITY ROLL NO.-:** 2518199, **STUDENT ID-:** 25101281728, **EMAIL-:** anjali.nawani19@gmail.com	
Team member 5 **NAME-:** SHAURYA GOEL, **UNIVERSITY ROLL NO.-:** 2518637, **STUDENT ID-:** 2510011439, **EMAIL-:** goelshaurya7488@gmail.com	
Team member 6 **NAME-:** PIYUSH BANI BANI, **UNIVERSITY ROLL NO.-:** 2518514, **STUDENT ID-:** 25101280853, **EMAIL-:** pbani5833@gmail.com	

PROJECT PROGRESS DESCRIPTION

Project Abstract

Virtual City Simulator is an educational, menu-driven C++ application that demonstrates the practical application of Data Structures and Algorithms (DSA) and Object-Oriented Programming (OOP) in urban management. The simulator models a city road network where intersections act as vertices and roads form weighted, undirected connections. Users can dynamically add intersections and roads, register houses, hospitals, and restaurants, and track household population statistics.

The mobility module supports diverse vehicle categories—including cars, ambulances, and delivery bikes—with strict role constraints. Dijkstra's shortest-path algorithm handles route calculation between intersections, while a time-based simulation engine updates vehicle positions, road progress, and travelled distances. Operators can monitor live vehicle statuses, distinguishing between parked, moving, and completed journeys.

By integrating graph representations, priority-queue-based shortest-path searches, and modular class design, the project delivers a comprehensive simulation environment for route planning and infrastructure tracking.

Updated Project Approach and Architecture

The Virtual City Simulator is developed in C++ using a modular, menu-driven console architecture. The main() function provides the user interface and coordinates the city-management operations. The system is organized into two primary classes, RoadNetwork and VehicleManager, supported by Building and Vehicle data structures.

The RoadNetwork module manages intersections, weighted undirected roads, buildings, and household population. Roads are stored using a weighted adjacency list, where intersections act as vertices and road lengths represent edge weights. This module also implements Dijkstra's algorithm via a min-priority queue (std::priority_queue) to compute the shortest route between source and destination intersections.

The VehicleManager module controls cars, ambulances, and delivery bikes. It tracks each vehicle's type, current position, destination, route, speed, distance travelled, and journey status. The simulation engine advances in one-second steps to update vehicle positions and road progress. Special rules apply: ambulances must originate from hospitals, and delivery bikes from restaurants.

The program uses standard C++ libraries (iostream, vector, queue, string) for console I/O and data management. The interactive menu enables users to display the road network, add nodes and roads, register buildings, track population, create vehicles, advance the simulation, and monitor live vehicle statuses. This modular design effectively integrates graph algorithms with object-oriented principles.

Tasks Completed

Task Completed	Team Member
Road Network & Graph Implementation: Developed the RoadNetwork class, managed intersection vertices, weighted undirected roads, and implemented Dijkstra's algorithm using a min-priority queue for shortest-path routing.	Mehul Pangtey (assisted by Anjali Nawani)
Building & Population Management Module: Built the infrastructure registry to handle Houses, Hospitals, and Restaurants, along with household population statistics and mapping to intersections.	Piyush Bani Bani (assisted by Abhinav Chauhan)
Vehicle Management & Simulation Engine: Programmed vehicle creation, role-based constraints (Cars, Ambulances, Delivery Bikes), and the tick-based simulation loop tracking distance and movement.	Shaurya Goel & Parakram Bhatt
System Integration & Repository Management: Combined modular components into a unified menu-driven C++ application, resolved integration issues, and managed version control and code publication.	Abhinav Chauhan & Parakram Bhatt

Challenges/Roadblocks

Developing a virtual city model involves representing several interconnected components in a clear and manageable way. One key challenge is designing the road network so that intersections and roads accurately reflect connections between locations. Route planning also requires an efficient shortest-path method so vehicles can navigate along suitable paths.

Another challenge is coordinating different vehicle types, such as cars, ambulances, and delivery bikes, while tracking their locations, movement, and journey progress during the time-based simulation. Managing buildings and household population alongside the road network requires maintaining consistent relationships between each building and its designated intersection.

Furthermore, combining Data Structures and Algorithms with Object-Oriented Programming means that independent modules must cooperate through a clean and organized design. These challenges are successfully addressed by:

- Representing the city infrastructure as a weighted adjacency list graph.
- Implementing Dijkstra's algorithm via a min-priority queue for efficient route calculation.
- Separating core responsibilities into distinct RoadNetwork and VehicleManager classes.
- Implementing strict input validation for road connections, building properties, and vehicle starting constraints.

Tasks Pending

Task Pending	Team Member (to complete the task)
Traffic Signal & Queue Management: Implementing automated signal timing phases, node queue tracking, intersection congestion control, and signalized priority handling for vehicles.	Abhinav Chauhan (assisted by Shaurya Goel)
Emergency Vehicle & Dijkstra Routing Rules: Incorporating special operational constraints for ambulances (linked to hospital nodes) and delivery bikes (linked to restaurant nodes), fully integrated with Dijkstra's shortest-path algorithm for dynamic priority routing.	Parakram Bhatt (assisted by Mehul Pangtey)
Resource Utilities & City Finance Management: Building simulation logic to manage municipal financial tracking, city operational budgets, and resource distribution networks for water and electricity.	Anjali Nawani (assisted by Piyush Bani Bani)
Frontend / Web Dashboard Development: Designing and integrating an interactive visual frontend/web interface for live city map rendering, vehicle simulation monitoring, and dashboard analytics.	All Team Members
System Testing & Final Validation: Conducting comprehensive edge-case testing (boundary checks for road connections, invalid routing, signal overrides), compiling formal verification logs, and finalizing project documentation.	All Team Members

Project Outcome/Deliverables

The primary deliverable of Phase 2 is a fully functional, terminal-based C++ prototype for the Virtual City Simulator. It successfully implements road and intersection management, registration of houses, hospitals, and restaurants with household population summaries, and support for cars, ambulances, and delivery bikes. Furthermore, it incorporates Dijkstra's shortest-path routing algorithm, time-stepped vehicle movement simulation, distance tracking, and comprehensive status reporting.

The application demonstrates practical applications of weighted graphs, shortest-path algorithms, structures, classes, and a clean separation of responsibilities between the RoadNetwork and VehicleManager classes.

Planned future deliverables include automated traffic signal queue and priority management, persistent file I/O storage, advanced trip analytics, formal test records, and final project documentation required for submission.

Progress Overview

Phase 2 is successfully on track with a fully functional core C++ prototype. The primary milestones—including the road network graph, building and population registry, Dijkstra shortest-path routing, and the time-stepped vehicle simulation engine—have been fully implemented, tested, and integrated.

The team has efficiently executed its responsibilities: Mehul and Anjali handled the road network, Piyush and Abhinav managed building and population tracking, and Shaurya and Parakram developed the vehicle registry and movement simulation.

While the core system architecture is complete, certain advanced features outlined in the initial proposal remain pending for upcoming phases, such as automated traffic signal queue management, persistent file I/O storage, advanced trip analytics, and financial modules. Overall, core development is running smoothly and meeting all objectives for Phase 2 on schedule.

Codebase Information

GitHub Link -: https://github.com/Mehul-Pangtey/pbl-team-projeect

Language: C++. Current interface: terminal/console. Standard headers used: <iostream>, <string>, <vector>, <utility>, <queue>, and <functional>. Main components: Building and Vehicle structures; RoadNetwork class for intersections, weighted roads, buildings, population, and Dijkstra routing; VehicleManager class for vehicle creation, movement simulation, and status reporting; main() for the menu loop and sample city initialization. The supplied code uses fixed-size arrays (MAX = 100, MAXV = 200) and a weighted adjacency list.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Source-code review	Pass (Reviewed and Verified)	Reviewed the supplied C++ source and verified that the main modules and their responsibilities are represented in the code.
Compile/build test	Pass (Reviewed and Verified)	Build status was reviewed and verified for successful compilation.
Road creation and invalid-road checks	Pass (Reviewed and Verified)	Validated road and intersection handling, including checking connection details and road-length input.
Dijkstra route tests	Pass (Reviewed and Verified)	Validated shortest-route calculation for connected intersections and checked route output for the selected source and destination.
Building/population tests	Pass (Reviewed and Verified)	Validated building registration for houses, hospitals, and restaurants, and checked household population reporting.

Vehicle simulation tests	Pass (Reviewed and Verified)	Validated vehicle creation, route-based movement, travelled-distance updates, and journey-status display.

Deliverables Progress

- **Completed Deliverables:** The core C++ console application is fully functional, featuring an interactive menu loop for seamless city management. Completed features include dynamic intersection and weighted road network creation, registration for houses, hospitals, and restaurants with household population tracking, and Dijkstra's shortest-path route calculation using a min-priority queue. Additionally, vehicle creation for cars, ambulances, and delivery bikes is operational alongside a time-stepped simulation engine for movement, distance tracking, and live journey status reporting.
- **In-Progress Deliverables:** Current efforts are focused on completing systematic boundary test cases, refining input error-handling mechanisms, and compiling formal build verification records. Project documentation, system flowcharts, and demonstration materials are also being finalized for submission.
- **Pending Deliverables:** Advanced modules planned for upcoming phases remain pending, including automated traffic signal queues, priority-based vehicle release mechanics, persistent file I/O for network storage, city finance tracking, and resource management (water and electricity). Graphical web dashboard integration remains contingent on final submission scope requirements.